Does the Robot Look Done? A Replay Benchmark for LLM Supervisors of Learned Robot Policies

Tom Wang Ranveer Singh Ty Lipscomb

COS 597C, Project 1 report, October 26, 2026

Code, data, results and exact commands: `https://github.com/tom05919/OrcaBench`

Abstract

A learned robot policy does not know when it has finished a task, so a language-model supervisor that watches the cameras has to decide when the task is done and how often to look, and every look costs time. We built a benchmark that measures these two skills separately and scores them automatically. It replays 168 recorded rollouts of three learned policies on seven RoboCasa [1] kitchen tasks. The supervisor's only actions, "wait N steps" and "done", do not change what the robot does, so a recording is exactly what a live run would show, and every model sees identical rollouts. A *probe* asks 450 single "is it done?" questions labelled by the simulator's success checker; *supervision sessions* score 168 episodes with a time-charged value against baselines that need no language model. We evaluated four open vision-language models at three requested reasoning levels (12 configurations). Done-detection balanced accuracy is 0.69–0.78; reasoning raises it by about 0.06 for two models and not detectably for the other two. No configuration schedules its looks as well as a perfect detector that looks at a fixed interval, and none has a higher mean value than a fixed timer. A requested reasoning level was not a reliable treatment: for three of the four models, two levels produced the same amount of reasoning.

1 Motivation

Vision-language-action policies such as GR00T [2] and $\pi_{0.5}$ [3] carry out household tasks quickly but unreliably, and they do not stop when they are finished: in our recordings the policy keeps acting until the step budget runs out, even after the simulator's checker has registered success. A natural remedy is a supervisor, a language model that periodically looks at the cameras and ends the episode when the task is done [4, 5]. Whether such a supervisor helps depends on two skills:

- **Done-detection:** recognising from camera images that the task is complete.
- **Look scheduling:** choosing how long to let the policy run before looking again. Each look costs seconds on the task clock and money, so looking constantly is not free and looking rarely wastes time after completion.

Existing evaluations either test success detection on curated clips [6] or report the end-to-end success of a combined LLM–robot system, where perception, planning and time cost are mixed into one number. Our own first benchmark did the latter and could not tell which skill failed (Section 5). This report describes the benchmark we rebuilt to measure the two skills separately, and its results on four models.

Contributions. (1) A replay benchmark that isolates done-detection (the probe) and look scheduling (the sessions), scored by the simulator's checker, with baselines that need no language model. (2) Results for four open models at three requested reasoning levels, with a check of whether each level changed the reasoning, 95% cluster bootstrap intervals, and full accounting of cost and losses. (3) An account of what did not work.

2 Benchmark design

Why replay. Once re-prompting is excluded (it did not help these policies, Section 5), the supervisor can only say "wait N steps" or "done". Neither changes the robot, so an uninterrupted recording of the policy contains everything the supervisor would have seen in a live run. Replay gives every model identical inputs (no robot randomness between conditions), makes runs repeatable and cheap, and needs no GPU.

Data. 168 recorded rollouts (two machines, 84 each): three policies from the official RoboCasa365 checkpoints (GR00T N1.5 multitask, GR00T N1.5 post-trained on 30% of the target-task demonstrations, and $\pi_{0.5}$ multitask), seven tasks and four kitchen scenes. Three tasks are *leave-alone* tasks the policies were trained on (OpenCabinet, PickPlaceCounterToStove, CloseFridge); four are unseen composite tasks. 57 rollouts reach success. Each rollout has three 256×256 camera videos (a frame every 10 control steps, 20 steps per second) and the checker's value at every step.

Probe (done-detection alone). One question is one model call with no history: the task instruction and three images (all three cameras now, 2 s earlier and 5 s earlier, tiled). The model answers `{"done": true}` or `false`. Positives are taken at the checker's first true frame and 5 s later; negatives 2 s and 10 s before it, and at the middle and end of rollouts that never succeed (114 positives, 336 negatives, fixed seed). The primary measure is balanced accuracy.

Sessions (detection and scheduling together). A session is one scene's 21 episodes (7 tasks $\times$ 3 policies) in a fixed order, with the policy identified only by an anonymous letter; 8 sessions give 168 episodes per configuration. In an episode the model starts at step 0 and repeatedly answers `wait` N (10–400 steps) or `done`; at each look it sees the cameras now and at two earlier points of the last interval, the step, the time left, the look cost and its earlier looks. "Done" is scored by the checker at that step; reaching the step budget is a failure. Before each episode it sees one line per earlier episode of the session (policy letter, task, outcome, looks), and nothing about the current episode's checker. *A real episode* (Gemma 4, low reasoning, CloseFridge, policy R): the model waited 100, 200

Decision	Reason
Replay recorded rollouts, not live runs	The supervisor's actions do not change the robot; identical inputs for every model, repeatable, no GPU.
Probe separate from sessions	Our live benchmark mixed skills in one score; the probe isolates perception.
Labels from the simulator checker; the 2 s just before success not probed	Automatic scoring; the checker lags the images by about 1 s, so frames in that band are ambiguous.
Balanced accuracy	Only 25% of probes are positive, mostly from three tasks; plain accuracy would reward always saying "not done".
Same look cost $c = 2$ s for every model	Measured latency on a shared API reflects serving load, not the model.
Baselines tuned in other scenes	A fixed rule must not be tuned on the rollout it is tested on.
Past outcomes shown, policies anonymous	Adapting to a policy is impossible without feedback; the current episode's outcome is never shown.
Same three levels for every model, recorded on every call; 32,768-token reply limit	A uniform design; recording the reasoning length checks that a level did something; a high limit avoids penalising long reasoning.
Levels compared within a model only	Providers map a requested level onto each model's own settings differently.
95% bootstrap clustered by rollout (probe) or task and scene (sessions)	Answers about the same rollout are correlated.

Table 1: Design decisions and their reasons.

and 200 steps, then said "done" at step 500; the checker had turned true at step 493, so the episode succeeded with 4 looks and scored $1 - (25\,\mathrm{s} + 4 \times 2\,\mathrm{s})/45\,\mathrm{s} = 0.27$.

Score and baselines. Value $=$ success $\times \max(0, 1 - (\text{robot seconds} + \text{looks} \times c)/\text{deadline})$, with the task's deadline and $c = 2$ s per look. Three baselines need no language model and pay for looks in the same way: a *fixed timer* declares at step T; a *perfect detector* looks every k steps and declares at the first look where the checker is true (the bar for scheduling); the *oracle* declares at the first success. T and k are tuned on the same policy and task *in the other scenes*. The primary session measure is the model's value minus the perfect detector's on the same recording, on the leave-alone tasks (72 episodes per configuration, 50 of them on rollouts that succeed).

Table 1 lists each design decision and the reason for it. All were fixed before the runs they govern.

3 Experimental setup

Models. Kimi K3, Gemma 4 31B, DeepSeek V4.1 Flash and GLM-5.3 Flash: the four models that accept images through Sail Research's synchronous API, which serves quantized weights. Results are for those deployments.

Reasoning levels. Every model ran at the same three requested levels, *none*, *low* and *high*, in both the probe and the sessions; GLM rejects disabling reasoning, so its "none" is the provider's *minimal* level. Every call recorded the length of the model's thinking, the stop reason, output tokens and attempts. Each configuration ran once: 450 probes and 168 session episodes. The grid cost an estimated $71.9 (Kimi K3 $58.3); everything run on the API for this study, including runs not analysed here, cost an estimated $138.7.

4 Results

4.1 Done-detection

Balanced accuracy lies between 0.69 and 0.78 in every configuration (Table 2, Figure 1). Turning reasoning on helps Kimi K3 (high − none $+0.064$ [$+0.013$, $+0.114$]) and DeepSeek V4.1 Flash ($+0.058$ [$+0.006$, $+0.109$]), mainly because the model says "done" more readily when the task is done: its hit rate rises from 0.64 to 0.85 (Kimi) and from 0.55 to 0.76 (DeepSeek). Gemma 4 does not change although its reasoning does (0 to about 840 characters), and GLM's only reasoning level adds $+0.032$ [-0.007, $+0.072$]. Low and high never differ. The errors concentrate just before success: every configuration says "done" on 62–80% of frames 2 s before the robot succeeds, where the task often looks finished, and on 36–50% of frames from leave-alone rollouts that never succeed.

4.2 Look scheduling

No configuration reaches the perfect detector: all 12 intervals for value minus detector lie below zero (Table 2, Figure 2). The best configurations (Gemma 4 with no reasoning, Kimi K3 at low) give up about 0.06 of the detector's 0.173. No configuration has a higher mean value than the fixed timer (0.144), and only one interval (Kimi K3 high) reaches it. Reasoning changes the trade-off rather than the outcome. Without it, Kimi K3 and DeepSeek look 9–10 times per episode and are right when they declare on 76–82% of the rollouts that succeed, but they pay for every look and declare late (median 10–18 s after success). With reasoning they look 4–7 times, but declare too early on 38–62% of the rollouts that succeed. On the 22 leave-alone rollouts that never succeed, every configuration eventually says "done"; these episodes score 0 either way. Unseen tasks add nothing: 7 of 96 of those rollouts succeed, and every value there is at most 0.008.

Adaptation. If a model adapted to the anonymous policies, the policy that usually finishes later should get the longer first wait and the later declaration. Within task blocks, only one of 24 such comparisons points that way (Gemma 4 none, first wait, from just 16 untied pairs). For DeepSeek low and all three GLM levels the slower policy was declared *earlier* more often than not, which fits early false "done" on slow policies that fail; we did not test that explanation. This test was declared exploratory and has little power (at most 64 policy pairs per configuration).

4.3 Is a reasoning level a treatment?

Only Kimi K3 has three distinct levels (median thinking 0, 343 and 1,576 characters). Gemma 4 and DeepSeek think the same amount at low and high, and GLM's low level produces no reasoning at all, exactly like its minimal level. A benchmark that varies "reasoning effort" through a provider's setting should therefore measure the reasoning it actually obtained, as we did, rather than assume the label.

5 What did not work

The first benchmark (shown on October 16). Language-model supervisors (Claude Opus 5.5 and Sonnet 5.5, and Kimi K3 later) supervised the same three policies live on GPU machines. They could wait, rewrite the policy's instruction, restart

		Done-detection probe (450 questions)			Sessions, leave-alone tasks (72 episodes, 2 s per look)		
Model	Level	Thinking (chars)	Balanced accuracy [95% CI]	Change from none [95% CI]	Value − perfect detector [95% CI]	Looks / episode	Correct (%)
Kimi K3	none	0	0.709 [0.664, 0.755]	—	−0.121 [−0.167, −0.077]	10.0	76
	low	343	0.771 [0.733, 0.809]	+0.062 [+0.006, +0.117]	−0.064 [−0.118, −0.012]	4.6	58
	high	1,576	0.772 [0.727, 0.814]	+0.064 [+0.013, +0.114]	−0.071 [−0.139, −0.023]	4.4	62
Gemma 4 31B	none	0	0.757 [0.719, 0.797]	—	−0.061 [−0.097, −0.028]	4.3	72
	low	838	0.763 [0.721, 0.804]	+0.006 [−0.022, +0.033]	−0.085 [−0.141, −0.032]	5.5	60
	high	844	0.752 [0.708, 0.793]	−0.006 [−0.039, +0.026]	−0.090 [−0.125, −0.057]	5.4	60
DeepSeek V4.1 Flash	none	0	0.689 [0.633, 0.744]	—	−0.151 [−0.221, −0.084]	9.3	82
	low	1,556	0.761 [0.716, 0.805]	+0.073 [+0.021, +0.127]	−0.125 [−0.182, −0.070]	7.0	38
	high	1,630	0.746 [0.691, 0.797]	+0.058 [+0.006, +0.109]	−0.096 [−0.155, −0.043]	6.6	58
GLM-5.3 Flash	none (min.)	0	0.751 [0.705, 0.796]	—	−0.130 [−0.197, −0.066]	7.7	60
	low	0	0.753 [0.704, 0.799]	+0.002 [−0.043, +0.044]	−0.109 [−0.166, −0.058]	8.3	62
	high	206	0.784 [0.739, 0.826]	+0.032 [−0.007, +0.072]	−0.125 [−0.183, −0.065]	7.9	56

Table 2: Results for every configuration. *Thinking*: median characters of the model's reasoning per probe answer, the check that a requested level changed the reasoning. *Change from none*: paired difference on the same probes. *Correct*: share of the 50 rollouts that succeed on which the model's "done" was scored a success (the alternative is a premature "done"). Baselines on the same leave-alone episodes: perfect detector 0.173, timer 0.144, oracle 0.284 (mean value). No reply was cut off by the reply limit.

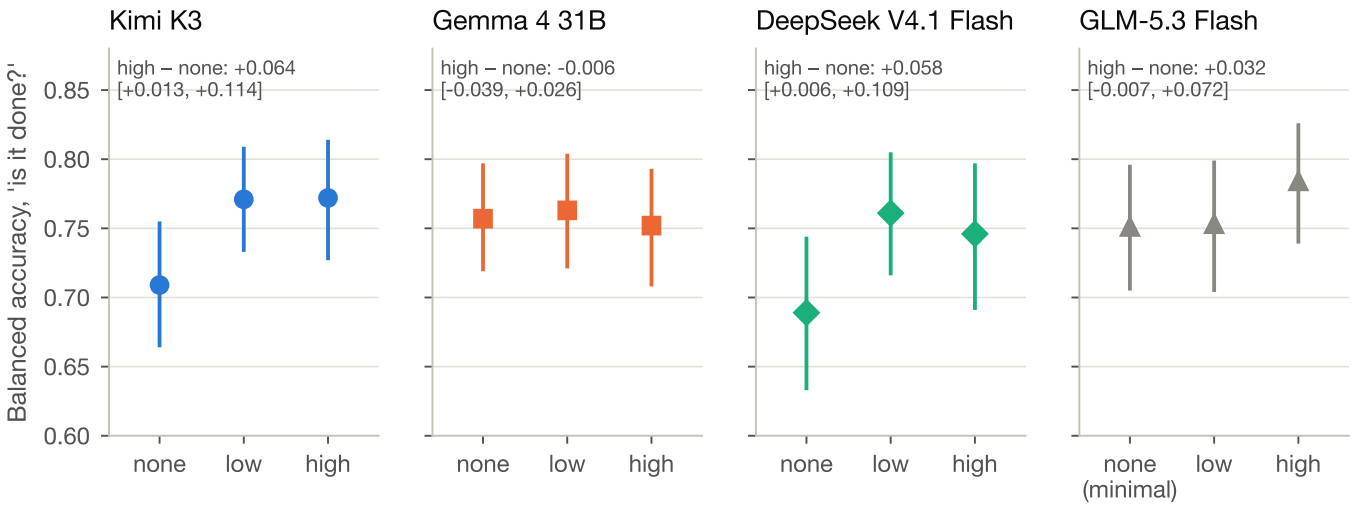


Figure 1: Done-detection: balanced accuracy by requested reasoning level, one panel per model (450 probes per point; 95% bootstrap CI clustered by recorded rollout; unreadable replies count as wrong). The text in each panel is the paired difference high − none on the same probes. GLM's "none" is the provider's minimal level. Levels are comparable within a panel, not across panels.

it, or declare done; every model call was charged its measured latency (4.7 s for Opus) on the task clock, and the headline was regret against a fixed declaration rule tuned in other scenes. Both models lost to the rule (regret 0.049 [0.004, 0.100] for Opus, 0.066 [0.016, 0.126] for Sonnet) and could not be told apart (−0.016 [−0.058, 0.028]). The number did not say *why*, for four reasons:

- **One score mixed three skills** (re-prompting, done-detection, look scheduling), so a loss could come from any of them. The rebuilt benchmark measures them separately.
- **Re-prompting could not help.** Even a privileged prompter that reads the simulator's state and gives correct step-by-step instructions solved 3 of 76 paired unseen-task episodes, against 4 of 76 for the unchanged policy. Supervisors that rewrote the instruction also failed more often (a small, unpaired comparison). We therefore removed re-prompting.
- **The latency charge decided the result.** In a post-hoc analysis with no charge per call, Opus and Kimi were statistically tied with the fixed rule. Latency on a shared API measures serving load, so the rebuilt benchmark charges every model the same 2 s per look.
- **Most episodes carried no signal.** The robot alone almost never succeeds on the unseen tasks, so 12 of 21 episodes per session scored about 0 for every supervisor and every rule. Live runs also gave each model different rollouts, one pass per model, and data spread over three machines.

Problems during this study, and their fixes. (1) The first replay runs sent no reasoning setting, so each model used the provider's undocumented default (Gemma does not reason by default; the others do), confounding model with reasoning. Those runs are excluded; the grid above declares and records the level. (2) A pilot of 10 probes was used to drop levels that did not change the reasoning; with so few probes it gave models different sets of levels (and was wrong for DeepSeek),

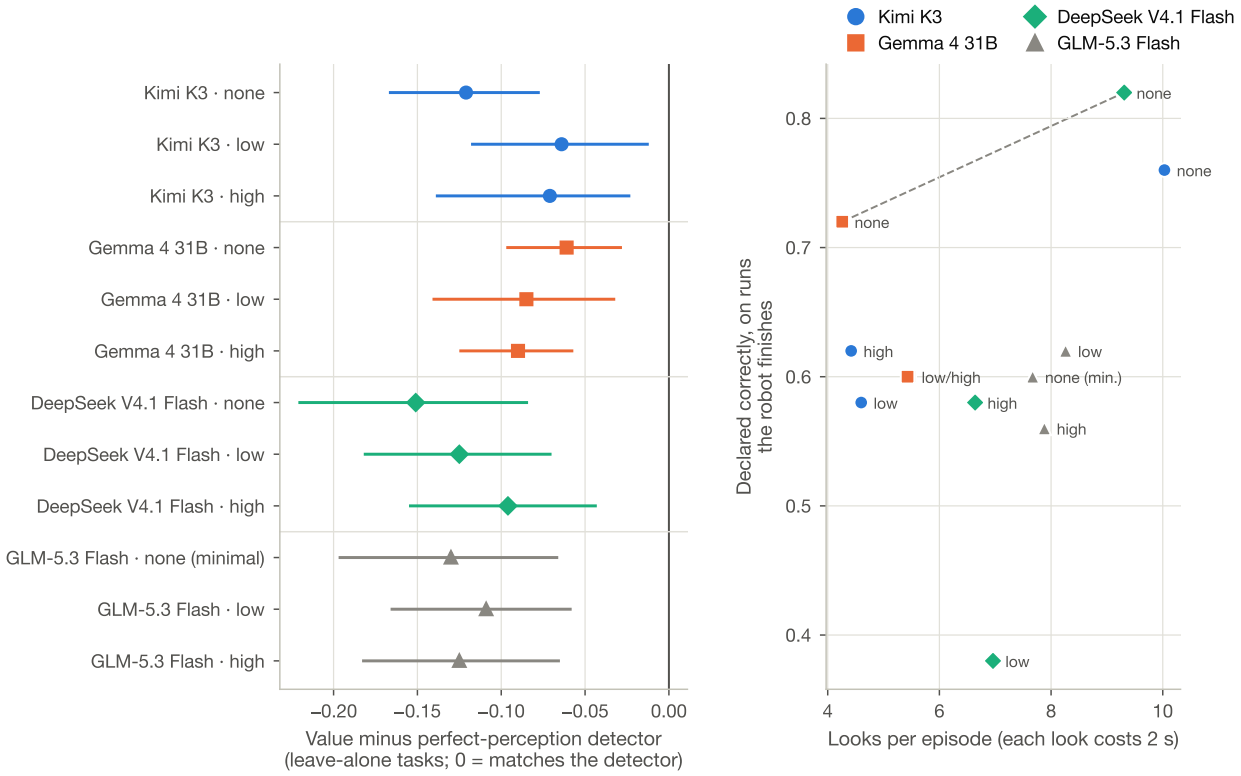


Figure 2: Look scheduling on the leave-alone tasks (OpenCabinet, PickPlaceCounterToStove, CloseFridge; 2 s per look). Left: value minus the perfect detector on the same rollouts, 95% bootstrap CI clustered by task and scene (0 = as good as the detector). Right: share of correct declarations on the rollouts that succeed, against looks per episode; the dashed line joins configurations that no other configuration beats on both axes.

so we replaced it with one grid for every model. (3) Our spending ledger booked every timed-out call at its worst case and stopped the runs at $72 of real spend; timeouts are now counted at the 99th percentile of real replies, and every call carries an idempotency key so a retry cannot be billed twice. Before the keys were added, 143 timed-out grid calls may have been billed twice. (4) Processes crashed 142 times on the provider's HTTP 404, 400 and 500 errors and on read timeouts, and the computer twice ran out of memory (whole videos were cached; frames are now decoded on demand). In the last configuration the provider held requests until they timed out and then reported them lost, and the laptop slept twice with its lid closed; progress fell from about 40 episodes an hour to 3–4, and we let it finish unchanged. Every look is saved when it arrives and an interrupted episode is replayed from its start, so every configuration is complete; in the grid, 34 probe replies and 9 session replies were paid for but lost, 181 episodes were interrupted and replayed (1,402 looks paid twice, $1.91), and requests in flight at a crash cannot be counted.

6 Limitations

Each configuration ran once, so the intervals cover sampling over rollouts, not run-to-run variation of the model. Reasoning levels are provider settings, not amounts of reasoning, and the models are quantized deployments. Samples are small: 57 of 168 rollouts succeed and positives come almost only from three tasks. Supervisors see images only, every 10 steps. The value score cannot distinguish a false "done" from running out of budget on a rollout that never succeeds. Finally, some "errors" just before success may be correct readings of images that the checker labels a second late.

Reproducing. The repository's README gives the commands; the recorded rollouts and every model reply are in its release `replay-data-2026-10-26`, so all tables and figures can be recomputed without API calls.

References

[1] S. Nasiriany et al. RoboCasa: Large-scale simulation of everyday tasks for generalist robots. RSS, 2024.

[2] NVIDIA. GR00T N1: An open foundation model for generalist humanoid robots. arXiv:2503.14734, 2025.

[3] Physical Intelligence. $\pi_{0.5}$: a vision-language-action model with open-world generalization. arXiv:2504.16054, 2025.

[4] M. Ahn et al. Do as I can, not as I say: Grounding language in robotic affordances. CoRL, 2022.

[5] W. Huang et al. Inner monologue: Embodied reasoning through planning with language models. CoRL, 2022.

[6] Y. Du et al. Vision-language models as success detectors. CoLLAs, 2023.